



Señales y Sistemas

Prácticas con MATLAB · 2.º Grado en Ingeniería Informática

Álvaro Lozano Murciego y André Sales Mendes

Universidad de Salamanca

ÍNDICE DE CONTENIDOS

1	Sesión 1. Introducción a MATLAB	3
1.1	Si vienes de C	3
1.2	El entorno MATLAB	4
1.3	Ejercicio 1. Variables escalares	5
1.4	Ejercicio 2. Vectores y matrices	7
1.5	Ejercicio 3. La función <code>size</code>	9
1.6	Ejercicios 4 y 5. Operaciones elemento a elemento	10
1.7	Ejercicio 6. Representación gráfica	11
1.8	Ejercicio 7. Bucle <code>for</code>	12
1.9	Ejercicio 8. Funciones en archivos <code>.m</code>	13
1.10	Referencia: señales elementales	14

Este libro reúne el material de prácticas de la asignatura **Señales y Sistemas** (2.º curso del Grado en Ingeniería Informática). Las prácticas se realizan con **MATLAB**, una herramienta pensada para el cálculo numérico y matricial que permite definir, representar y transformar señales con muy pocas líneas de código.

Cómo usar este libro

Cada sesión de prácticas tiene su propio capítulo. En cada uno encontrarás:

- **Objetivos de aprendizaje** al principio, para saber qué deberías dominar al terminar.
- **Explicaciones y ejemplos de código** listos para copiar en MATLAB y modificar.
- **Enunciados de los ejercicios**, resaltados en cajas, en el mismo orden que el guion de la sesión.
- **Soluciones propuestas** plegadas: intenta resolver cada ejercicio antes de desplegarlas.

Si ya programas en C

La primera sesión está escrita pensando en quien ya conoce C. Muchas explicaciones comparan directamente la sintaxis de ambos lenguajes para que aproveches lo que ya sabes.

Sesiones

- *Sesión 1. Introducción a MATLAB*

SESIÓN 1. INTRODUCCIÓN A MATLAB

Esta es la guía de referencia de la primera sesión de prácticas. Si ya sabes programar en C, verás que MATLAB tiene una sintaxis diferente, pero la lógica es familiar. Usa esta página junto con el guion de ejercicios: cada apartado corresponde a uno de ellos.

Objetivos de aprendizaje

- Reconocer las diferencias de sintaxis más importantes entre C y MATLAB.
- Manejar el entorno: consola, *Workspace*, editor de scripts y carpeta actual.
- Definir escalares, números complejos, vectores y matrices, y acceder a sus elementos.
- Distinguir las operaciones matriciales (`*`, `/`, `^`) de las operaciones elemento a elemento (`.*`, `./`, `.^`).
- Representar señales con `plot` y etiquetar correctamente las gráficas.
- Escribir bucles `for` y funciones en archivos `.m`.

1.1 Si vienes de C

MATLAB no es un lenguaje de propósito general: está diseñado para el cálculo matricial y numérico. Muchas cosas que en C requieren librerías y bucles aquí se escriben en una sola línea. El precio es que la sintaxis tiene varias diferencias que generan errores al principio. La [Tabla 1.1](#) resume las que más vas a notar el primer día.

Tabla 1.1: Equivalencias básicas entre C y MATLAB.

En C escribes...	En MATLAB es...	¿Por qué cambia?
<code>int a = 5;</code>	<code>a = 5;</code>	No se declaran tipos. Todo es <code>double</code> por defecto.
<code>// comentario</code>	<code>% comentario</code>	<code>%</code> inicia un comentario. El resto de la división se calcula con <code>mod(a, b)</code> .
<code>printf("x=%d\n", x);</code>	<code>disp(x)</code> o simplemente <code>x</code>	Sin <code>;</code> al final, MATLAB muestra el resultado automáticamente. También existe <code>fprintf</code> , casi idéntico a <code>printf</code> .
<code>v[0] = 1;</code>	<code>v(1) = 1;</code>	Los índices empiezan en 1, no en 0, y se escriben entre paréntesis.
<code>a * b</code> (dos números)	<code>a .* b</code> (dos vectores)	<code>*</code> es el producto matricial. Para multiplicar elemento a elemento se usa <code>.*</code> .
<code>#include <math.h></code>	(nada)	No hay <i>includes</i> : <code>sin</code> , <code>cos</code> , <code>exp</code> , <code>sqrt</code> ... están siempre disponibles.
<code>}</code> (cierra bloque)	<code>end</code>	Los bloques <code>for</code> , <code>if</code> , <code>while</code> y <code>function</code> se cierran con <code>end</code> .
<code>;</code> (termina la sentencia)	<code>;</code> (suprime la salida)	El <code>;</code> no es obligatorio: lo omites cuando quieres ver el valor en la consola.

El error más frecuente del primer día

Usar `*` en lugar de `.*` al operar dos vectores. Si MATLAB responde con `Error using * Incorrect dimensions for matrix multiplication`, ese es el problema.

Los dos fragmentos siguientes hacen lo mismo: rellenar un vector con los números del 1 al 10. Fíjate en el índice inicial, en la cabecera del `for` y en el `end` que sustituye a la llave de cierre.

Código 1.1: Bucle `for` en C.

```
int v[10];
for (int k = 0; k < 10; k++) {
    v[k] = k + 1;
}
```

Código 1.2: El mismo bucle en MATLAB.

```
v = zeros(1, 10);
for k = 1:10
    v(k) = k;
end
```

1.2 El entorno MATLAB

Al abrir MATLAB verás varias zonas de trabajo:

- **Command Window (consola):** escribes una orden, pulsas Intro y ves el resultado al momento.
- **Workspace:** lista las variables que existen en memoria, con su tamaño y su valor.
- **Editor:** donde escribes *scripts* y funciones en archivos `.m`.

- **Current Folder:** la carpeta de trabajo actual. MATLAB solo encuentra tus archivos `.m` si están en ella (o en el *path*).

Hay unos pocos comandos que usarás constantemente para mantener limpio el entorno:

Código 1.3: Comandos esenciales de limpieza.

```
clc           % limpia la consola (como cls en Windows)
clear        % borra todas las variables del Workspace
clear a b    % borra solo a y b
who          % lista las variables que existen ahora
whos         % lista las variables con su tipo y tamaño

% Buen hábito: empezar cada script con
clc; clear;
```

Script o consola

Puedes escribir directamente en la consola o guardar el código en un archivo `.m` y ejecutarlo con **F5** (o el botón *Run*). En esta sesión usa *scripts*: así puedes corregir y volver a ejecutar fácilmente.

1.3 Ejercicio 1. Variables escalares

Enunciado

Definir variables, con cualquier nombre, que almacenen los valores indicados en la tabla del guion: números, fracciones, raíces, funciones trigonométricas, logaritmos y números complejos.

1.3.1 Asignación

La asignación es igual que en C (`variable = valor`), con dos diferencias: no hay tipos y el `;` final solo sirve para que MATLAB no imprima el resultado.

Código 1.4: Asignación y constantes predefinidas.

```
a = 3;           % silencioso
b = -5.7;        % silencioso
b = -5.7         % sin ; -> muestra "b = -5.7000" en la consola

% Constantes predefinidas (no las uses como nombres de variables)
c = pi;          % 3.14159...
d = exp(1);      % número e = 2.71828...
```

1.3.2 Fracciones y raíces

MATLAB evalúa $*$ y $/$ de izquierda a derecha, igual que C. Por eso $5/2*7$ vale $(5/2) \cdot 7 = 17.5$ y no $5/14$. Usa paréntesis siempre que escribas una fracción con varios factores en el denominador.

Código 1.5: Fracciones, raíces y potencias.

```
f1 = 5 / (2*7);    % 5/14 = 0.3571
f2 = 1/3;          % 0.3333...

r1 = sqrt(2);      % raíz cuadrada de 2 = 1.4142...
r2 = 2^(1/3);      % raíz cúbica de 2: ^ es la potencia (pow en C)
r3 = 8^(2/3);      % = 4
```

1.3.3 Trigonometría

Las funciones trigonométricas trabajan **siempre en radianes**, igual que las de `math.h`.

Código 1.6: Funciones trigonométricas y sus inversas.

```
s1 = sin(pi/4);    % 0.7071
s2 = cos(pi/3);    % 0.5
s3 = tan(pi/4);    % 1

s4 = asin(0.5);    % pi/6
s5 = atan2(1, 1);  % pi/4 (arcotangente con cuadrante correcto)
```

1.3.4 Logaritmos y exponencial

Código 1.7: Logaritmos en distintas bases y exponencial.

```
l1 = log(exp(1));    % logaritmo natural: ln(e) = 1
l2 = log10(100);    % logaritmo en base 10 -> 2
l3 = log2(8);        % logaritmo en base 2 -> 3
e1 = exp(-2);        % e^(-2) = 0.1353
```

1.3.5 Números complejos

Los números complejos son imprescindibles en Señales y Sistemas. Un complejo puede escribirse en forma rectangular, $z = a + jb$, o en forma polar, $z = r e^{j\theta}$, donde $r = |z|$ es el módulo y θ el argumento. En MATLAB la unidad imaginaria se escribe `i` o `j`; la forma más segura es `1i` o `1j`.

Código 1.8: Definición y operaciones con números complejos.

```
z1 = 3 + 4i;          % forma rectangular
z2 = 2*exp(1i*pi/3);  % forma polar: r = 2, theta = pi/3

modulo = abs(z1);      % |z1| = 5
fase = angle(z1);      % argumento = atan2(4, 3) = 0.9273 rad
conjug = conj(z1);     % conjugado: 3 - 4i
parte_r = real(z1);    % parte real: 3
parte_i = imag(z1);    % parte imaginaria: 4
```

No uses `i` ni `j` como contadores

`i` y `j` representan la unidad imaginaria. Si los usas como variables de bucle, dejarán de valer $\sqrt{-1}$ y tus cálculos con complejos darán resultados erróneos sin ningún aviso. Usa `k`, `n` o `m` para los índices.

1.4 Ejercicio 2. Vectores y matrices

Enunciado

Definir los vectores y matrices indicados. Para los tres últimos (vectores muy largos) es **obligatorio** usar la notación con el operador `:`; no se pueden escribir todos los elementos a mano.

1.4.1 Definición explícita

Los elementos se escriben entre corchetes. El espacio (o la coma) separa columnas y el punto y coma separa filas.

Código 1.9: Vectores fila, vectores columna y matrices.

```
% Vector fila: elementos separados por espacios o comas
v = [1 2 3 4 5];
v = [1, 2, 3, 4, 5];    % equivalente

% Vector columna: elementos separados por punto y coma
c = [1; 2; 3; 4; 5];
```

(continues on next page)

(continued from previous page)

```
% Matriz 3x3: las filas se separan con ;  
A = [1 2 3; 4 5 6; 7 8 9];  
  
% Transpuesta: el operador ' convierte fila <-> columna  
ct = c'; % columna -> fila
```

1.4.2 El operador :

El operador `:` (*colon*) genera vectores con un patrón regular. Su sintaxis es `inicio:paso:fin`; si omites el paso, vale 1. El paso puede ser decimal o negativo.

Código 1.10: Vectores generados con el operador `:` y con `linspace`.

```
v1 = 1:10; % [1 2 3 4 5 6 7 8 9 10]  
v2 = 0:0.5:3; % [0 0.5 1 1.5 2 2.5 3]  
v3 = 10:-1:1; % cuenta atrás: [10 9 8 ... 1]  
v4 = 0:2:20; % pares de 0 a 20  
  
% linspace(a, b, N): N puntos equiespaciados entre a y b  
t = linspace(0, 10, 1000); % 1000 puntos en [0, 10]  
  
% En Señales, así se define el eje temporal  
t = 0:0.001:10; % de 0 a 10 con paso 0.001 (10001 puntos)
```

Cuándo usar `:` y cuándo `linspace`

Usa `inicio:paso:fin` cuando te importe el **paso** (por ejemplo, el periodo de muestreo). Usa `linspace(a, b, N)` cuando te importe el **número de puntos**. Ambos generan vectores fila.

1.4.3 Acceso a los elementos

Recuerda que el primer elemento tiene índice 1. La palabra `end` dentro de un índice significa «el último».

Código 1.11: Indexación de vectores y matrices.

```

v = [10 20 30 40 50];

v(1)          % primer elemento: 10    (en C sería v[0])
v(3)          % tercer elemento: 30    (en C sería v[2])
v(end)        % último elemento: 50
v(2:4)        % elementos 2, 3 y 4: [20 30 40]
v(end-1:end)  % penúltimo y último: [40 50]

% Matrices: A(filas, columna)
A(1, 2)       % fila 1, columna 2
A(2, :)       % toda la fila 2
A(:, 3)       % toda la columna 3

```

1.5 Ejercicio 3. La función size

Enunciado

Obtener el número de filas y de columnas de una matriz y de un vector fila mediante la función `size`, y almacenar esos valores en dos variables.

La función `size` devuelve las dimensiones de una variable. Llamada como `[m, n] = size(A)` devuelve a la vez filas y columnas; llamada como `size(A, dim)` devuelve solo una dimensión (`dim = 1` para filas, `dim = 2` para columnas). Para vectores, `length(v)` da directamente el número de elementos.

Código 1.12: Tres formas de consultar dimensiones.

```

A = [1 2 3; 4 5 6];          % matriz 2x3

% Forma 1: filas y columnas a la vez (lo que pide el ejercicio)
[filas, cols] = size(A);    % filas = 2, cols = 3

% Forma 2: una sola dimensión
filas = size(A, 1);          % número de filas
cols  = size(A, 2);          % número de columnas

% Forma 3: para vectores, length es más cómodo
v = 1:10;
n = length(v);               % n = 10
n = numel(v);                % número total de elementos

```

Comprueba tu resultado

Para un vector fila como `v = 1:10`, `size(v)` devuelve `[1 10]`: una fila y diez columnas. Si al transponerlo (`v'`) obtienes `[10 1]`, has entendido la diferencia entre vector fila y vector columna.

1.6 Ejercicios 4 y 5. Operaciones elemento a elemento

Enunciado

- **Ejercicio 4.** Multiplicar elemento a elemento dos vectores usando `.*`.
- **Ejercicio 5.** Dividir elemento a elemento con `./` y elevar al cuadrado cada elemento con `.^`.

Esta es una de las diferencias más importantes respecto a C. En MATLAB, `*` es la multiplicación **matricial**. Para operar dos vectores posición a posición se añade un punto delante del operador, como resume la [Tabla 1.2](#).

Tabla 1.2: Operadores elemento a elemento y su equivalente en C.

Operador	Acción	Equivalente «a mano» en C
<code>.*</code>	$c(k) = a(k) * b(k)$ para cada k	for ($k=0$; $k<n$; $k++$) $c[k] = a[k]*b[k]$;
<code>./</code>	$c(k) = a(k) / b(k)$ para cada k	for ($k=0$; $k<n$; $k++$) $c[k] = a[k]/b[k]$;
<code>.^</code>	$c(k) = a(k)^p$ para cada k	for ($k=0$; $k<n$; $k++$) $c[k] = \text{pow}(a[k], p)$;
<code>*</code>	Producto matricial (otro concepto)	No es lo mismo: exige que el número de columnas de a coincida con el número de filas de b .

Solución propuesta de los ejercicios 4 y 5

```
v = [1 2 3 4 5];
z = [2 4 1 3 6];

% Ejercicio 4: producto elemento a elemento
prod_ev = v .* z;          % [2 8 3 12 30]

% Ejercicio 5a: división elemento a elemento
div_ev = v ./ z;           % [0.5 0.5 3 1.3333 0.8333]

% Ejercicio 5b: cuadrado de cada elemento de v
cuadrado = v .^ 2;         % [1 4 9 16 25]
```

¿Por qué importa esto en Señales? Porque una señal en MATLAB es un vector con miles de muestras. Para calcular, por ejemplo, el seno amortiguado $y(t) = \sin(2\pi t) e^{-t}$ hay que multiplicar muestra a muestra:

Código 1.13: Seno amortiguado: producto elemento a elemento de dos señales.

```
t = 0:0.001:10;
y = sin(2*pi*t) .* exp(-t);
```

Cuidado

`sin(t) * exp(-t)` produce el error `Incorrect dimensions for matrix multiplication`, porque intenta multiplicar dos vectores fila como matrices. Siempre `.*` cuando operas dos vectores punto a punto. Multiplicar un vector por un **escalar** (`2*t`) sí funciona con `*`.

1.7 Ejercicio 6. Representación gráfica

i Enunciado

Representar gráficamente varias funciones en $t \in [0, 10]$ (la última en $t \in [-10, 10]$) usando `plot`. Usar `figure` para mostrarlas en ventanas separadas.

El patrón es siempre el mismo:

1. Definir el vector `t` con el dominio que pide el ejercicio y un paso pequeño (0.001 da buena resolución).
2. Calcular la señal con operaciones elemento a elemento sobre `t`.
3. Abrir una ventana nueva con `figure` y dibujar con `plot(t, y)`.
4. Añadir etiquetas. Una gráfica sin título ni ejes etiquetados es una gráfica incompleta.

Código 1.14: Patrón completo para representar una señal.

```
% Paso 1: eje temporal
t = 0:0.001:10;

% Paso 2: señal
y = sin(2*pi*t);

% Paso 3: abrir ventana y dibujar
figure;           % nueva ventana (evita sobrescribir la gráfica anterior)
plot(t, y);

% Paso 4: etiquetar
xlabel('t (s)');
ylabel('y(t)');
title('Señal senoidal');
grid on;
```

Las siguientes señales son ejemplos típicos del tipo de funciones que pide el ejercicio. Adáptalas a las expresiones concretas de tu guion.

Código 1.15: Señales de ejemplo para el ejercicio 6.

```
t = 0:0.001:10;
t2 = -10:0.001:10;      % dominio [-10, 10] para la última función

% Exponencial decreciente
figure; plot(t, exp(-t));
title('e^{-t}'); xlabel('t'); grid on;

% Coseno amortiguado
figure; plot(t, exp(-0.5*t) .* cos(2*pi*t));
title('e^{-0.5t} cos(2\pi t)'); xlabel('t'); grid on;

% Señal cuadrada (signo de un seno)
figure; plot(t, sign(sin(pi*t)));
title('Cuadrada'); xlabel('t'); grid on;

% Sinc en [-10, 10]: sinc(x) = sin(pi x)/(pi x)
figure; plot(t2, sinc(t2));
title('sinc(t)'); xlabel('t'); grid on;
```

i Si sinc no existe en tu instalación

La función `sinc` pertenece a la *Signal Processing Toolbox*. Si no la tienes, calcula $y = \sin(\pi t^2) / (\pi t^2)$; y corrige el punto $t = 0$, donde la división da NaN, con $y(t^2 == 0) = 1$;

Algunas opciones de `plot` que te ahorrarán tiempo:

Código 1.16: Estilos de línea, varias curvas y rango de los ejes.

```
% Estilos: 'color' + 'línea' + 'marcador'
plot(t, y, 'b-');      % azul, línea continua
plot(t, y, 'r--');     % rojo, línea discontinua
plot(t, y, 'go');      % verde, círculo en cada punto

% Varias curvas en la misma figura
figure;
hold on;
plot(t, sin(t));
plot(t, cos(t));
legend('sin(t)', 'cos(t)');
hold off;

% Ajustar el rango de los ejes
xlim([0 10]);
ylim([-1.5 1.5]);
```

i Notación matemática en los títulos

En `title`, `xlabel` y `ylabel` puedes usar notación TeX básica: `'e^{-t}'` muestra un exponente, `'\pi'` muestra la letra griega y `'f_0'` muestra un subíndice.

1.8 Ejercicio 7. Bucle `for`

i Enunciado

Mediante un bucle `for`, crear un vector formado por los números naturales del 1 al 10.

El `for` de MATLAB es diferente al de C: no controla un contador con condición e incremento, sino que **recorre los elementos de un vector**. En cada vuelta, la variable del bucle toma el siguiente valor del vector.

Código 1.17: Estructura general del bucle `for`.

```
for variable = vector_de_valores
    % bloque de instrucciones (sin llaves)
end
```

Algunos vectores de iteración habituales (solo se muestran las cabeceras):

Código 1.18: Ejemplos de cabeceras de bucle.

```

for k = 1:10           % k = 1, 2, ..., 10
for k = 0:2:20         % k = 0, 2, 4, ..., 20
for k = 10:-1:1        % k = 10, 9, 8, ..., 1
for x = [3 7 -1]       % x = 3, luego 7, luego -1

```

i Solución propuesta del ejercicio 7

```

v = zeros(1, 10);      % preasignar el vector
for k = 1:10
    v(k) = k;
end
disp(v)                % 1 2 3 4 5 6 7 8 9 10

```

i Preasignación

Antes de un bucle que rellena un vector, créalo con `zeros(1, n)`. Sin preasignación, MATLAB redimensiona el vector en cada iteración: funciona, pero es mucho más lento con vectores grandes.

1.9 Ejercicio 8. Funciones en archivos .m

i Enunciado

Escribir una función MATLAB que devuelva el producto de todos los elementos de un vector que se pasa como argumento.

Una función MATLAB se guarda en un archivo `.m` cuyo nombre debe ser **exactamente igual** al nombre de la función. La primera línea define las salidas, el nombre y las entradas: `function salida = nombre(entrada)`. No hay `return` con valor: la función devuelve lo que valga la variable de salida al llegar al `end`.

Compara la versión en C con la versión en MATLAB. En MATLAB no hace falta pasar el tamaño del vector, porque la función puede averiguarlo con `length`.

Código 1.19: Producto de los elementos de un vector en C.

```

double producto(double *v, int n) {
    double r = 1.0;
    for (int k = 0; k < n; k++)
        r *= v[k];
    return r;
}

```

i Solución propuesta: archivo producto_elementos.m

```

function resultado = producto_elementos(v)
% PRODUCTO_ELEMENTOS Devuelve el producto de todos los elementos de v.
%

```

```
% resultado = producto_elementos(v)
%
% Entrada: v - vector numérico (fila o columna)
% Salida: resultado - escalar con el producto acumulado

resultado = 1; % elemento neutro del producto
for k = 1:length(v)
    resultado = resultado * v(k);
end
end
```

Una vez guardado el archivo, la función se llama desde la consola o desde cualquier *script*:

Código 1.20: Llamadas a la función y comprobación con `prod`.

```
p = producto_elementos([1 2 3 4 5]); % p = 120
p = producto_elementos(1:10); % p = 3628800 = 10!

% Comprobación: MATLAB incluye la función prod
prod([1 2 3 4 5]) % = 120
```

MATLAB no encuentra mi función

El archivo `producto_elementos.m` debe estar en la carpeta actual (*Current Folder*). Comprueba dónde estás con `pwd` y cambia de carpeta con `cd` o desde la barra de direcciones de MATLAB. Comprueba también que el nombre del archivo coincide con el de la función.

Una función puede devolver varias salidas. Se declaran entre corchetes y se recogen igual:

Código 1.21: Función con dos salidas (archivo `mi_max.m`).

```
function [mx, idx] = mi_max(v)
% Devuelve el máximo de v y su posición
[mx, idx] = max(v);
end
```

Código 1.22: Llamadas a una función con varias salidas.

```
[m, n] = mi_max([3 8 2 5]); % m = 8, n = 2
m = mi_max([3 8 2 5]); % solo el máximo
```

1.10 Referencia: señales elementales

Estas son las señales que más aparecerán a lo largo del curso, listas para copiar y adaptar. El escalón unitario, el pulso rectangular de anchura a y la rampa se definen como

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad \Pi_a(t) = \begin{cases} 1, & |t| < a/2 \\ 0, & \text{en otro caso} \end{cases} \quad r(t) = t u(t) \quad (1.1)$$

En MATLAB, las expresiones de (1.1) se construyen con comparaciones: $(t \geq 0)$ vale 1 donde se cumple la condición y 0 donde no.

Código 1.23: Señales elementales construidas con comparaciones lógicas.

```

t = -5:0.001:5;

% Escalón unitario u(t)
u = (t >= 0);

% Pulso rectangular de anchura a centrado en 0
a = 2;
rect = (t >= -a/2) & (t < a/2);

% Rampa r(t) = t u(t)
ramp = t .* (t >= 0);

% Seno amortiguado causal (respuesta típica de un sistema de 2.º orden)
y_am = exp(-0.5*t) .* sin(2*pi*t) .* (t >= 0);

```

Con esas señales puedes calcular propiedades y aplicar transformaciones básicas. La energía de una señal, $E = \int |x(t)|^2 dt$, se aproxima numéricamente sumando las muestras al cuadrado y multiplicando por el paso Δt .

Código 1.24: Energía, desplazamiento e inversión temporal.

```

dt = t(2) - t(1); % paso de muestreo

% Energía de la señal (integración numérica)
E = sum(abs(y_am).^2) * dt;

% Desplazamiento temporal: y(t - t0)
t0 = 1;
y_ret = exp(-0.5*(t-t0)) .* sin(2*pi*(t-t0)) .* (t >= t0);

% Inversión temporal: y(-t)
y_inv = exp(0.5*t) .* sin(-2*pi*t) .* (t <= 0);

```

Resumen de la sesión

- En MATLAB no se declaran tipos, los índices empiezan en 1 y los bloques se cierran con end.
- Los vectores se crean con [], con inicio:paso:fin o con linspace.
- Entre dos vectores, usa siempre .*, ./ y .^.
- Toda gráfica lleva figure, plot, título y ejes etiquetados.
- Una función vive en un archivo .m con su mismo nombre, en la carpeta actual.